



Tecnológico de Monterrey

Programación de estructuras de datos y algoritmos fundamentales (Gpo 607)

Gabriela Michelle Lagos Aguilar - A01648192

20/09/2026

Importancia de ordenar y buscar en este tipo de logs

Un log así de tedioso, con una gran densidad de datos de tiempo, IP y errores, es casi imposible navegar sin tener un método de ordenamiento y búsqueda. En este caso, la búsqueda se enfoca alrededor del tiempo y fecha que se registraron errores. Con ese método de búsqueda, se le pueden dar uso a los datos, como, por ejemplo, buscar patrones de errores alrededor de fechas o analizar clusters de errores. Además, sin ordenamiento, no se podría hacer búsquedas eficientes como con los métodos que hemos ocupado. Es decir, el ordenamiento es necesario para desenredar los datos para hacerlos leíbles en la búsqueda, que es cuando verdaderamente se le da un uso práctico a los datos.

Comparación conceptual de algoritmos

Los algoritmos elegidos fueron: swap sort, bubble sort, selection sort, insert sort, merge sort, quick sort y shell sort. Teóricamente, los algoritmos que no sufrirían mucho cambio entre las dos bases de datos, serían Merge Sort y Selection Sort. El Merge Sort tiene que dividir la base de datos y luego swapear las veces necesarias sin importar el orden de datos. El hecho de que están casi ordenados puede causar, numéricamente, una diferencia, pero no será gigante. El Selection Sort es similar en que tiene que completar su proceso sin importar el orden; siempre va a buscar el menor en toda la lista, por lo que no debería de haber gran cambio.

Los que pienso yo que mostrarán mucho mejor empeño en la lista de casi ordenados son swap Sort e Insert Sort. Éste último, el Insert Sort, chequea que el dato presente no sea más chico que el que tiene a la izquierda, pero si ya están organizados, puede recorrer la lista con más rapidez. Swap Sort pienso que tendrá un cambio por la cantidad de veces que se ahorraría intercambiar en una lista que ya casi está ordenada.

Quick sort será probablemente el único que vaya más lento en los datos casi ordenados, ya que empieza con el último dato que será, en el caso de la casi ordenada, uno de los más grandes, entonces no tendrá mucho si es que nada para pasar a su derecha.

Interpretación de mediciones

Corrí cada uno con las dos databases de logs para comparar su desempeño. La primera base de datos viene completamente desordenada, mientras que la otra viene casi ordenada. Incluyo también una estimación mía de cuánto se iba a tardar.

Algoritmo	Desordenados (ms)	Casi ordenados (ms)	Predicción desordenados (ms)	Predicción casi ordenados (ms)
-----------	-------------------	---------------------	------------------------------	--------------------------------

Swap sort	7051	276	1000	2000
Bubble sort	8361	2749	3000	1000
Selection sort	312	261	300	100
Insert sort	6204	14	7000	2000
Merge sort	57	53	1000	30
Quick sort	3802	2789	1000	1000
Shell sort	54	46	300	100

Además de comprobar que la predicción humana no es muy certera, se puede comprobar que algunos algoritmos son mejores pero un tipo de datos que otros. La diferencia que más me tomó de sorpresa fue la se Swap Sort, mostrando una buena eficiencia para datos casi ordenados y uno de los peores desempeños para los datos completamente desordenados. Esto viene por la manera que cada uno analiza los datos y los reorganiza para llegar al orden. Swap Sort tiene que, en cada iteración, comparar un par de números y empujar uno al final, por lo que toma mucho tiempo en datos completamente desordenados. Muestra mejor desempeño en los casi ordenados porque tiene que hacer menos swaps cada iteración, y es probable que los datos mayores estén cerca del final, facilitando el proceso. Todos mostraron un notable mejor desempeño en la segunda base de datos, casi ordenada, aunque unos más que otros.

Uso de IA

Aunque creo haberlo puesto en mi código, yo nunca ocupo el Copilot integrado en Visual Studio Code. De hecho, tengo algunas herramientas de auto completado apagadas también, como aquella que te muestra el nombre de las variables cuando las empiezas a escribir. Así me tocó aprender con mi anterior maestro y francamente no me quiero mal acostumbrar a apoyarme en algunas de estas herramientas cuando estoy activamente escribiendo código. Sin embargo, yo sí ocupo la IA en códigos donde puede haber un riesgo en correrlo sin checarlo bien, como podría ser el caso con memory leaks. Dicho esto, los prompts que ocupé directamente en Gemini fueron:

1. Revisa el código minuciosamente y dime si tiene fallas de lógica o bugs.
2. Revisa el código. Qué cambios son necesarios de hacer para ocuparlo con una base de datos grande.

El segundo prompt me dio mucho que buscar en la tarea pasada de la base de datos al azar de 100,000 números. En ese caso ya ni siquiera corrí todos los números por puro miedo de que algo pasara mal. Tomando en cuenta que yo reciclo mucho mi código, en efecto, ocupé los mismos Sorts del archivo de la semana pasada, el segundo prompt me ayudó a manejar el tema de `.clear()` en algunas funciones.

Luego, algo que marqué en mi código y que es importante mencionar, es que hice un búsqueda en Google para la parte de búsqueda por rango, y terminé sacando la idea que ocupé de la burbuja de IA que se pone como el primer resultado en la plataforma. La idea es agregar a la fecha mínima un "000.000.000.000" de IP y una "A", mientras que a la fecha final un "999.999.999.999" y Z. En la burbuja sugería agregar datos que pudieran abarcar correctamente los datos de en medio. No puede haber un dato menor a un IP de ceros y un error que solo sea "A", y viceversa para el caso de fecha final.

De las sugerencias que me da, trato de seguirlas por el bien del código, pero hubo una vez que me recomendó ocupar "int opcion opcionValida=1", lo que a mí no me hizo sentido si se podría ocupar un valor booleano y evitar hacer la matemática que había planteado. Ignoré la sugerencia y la siguiente vez que le mandé el código para arreglar un desorden de corchetes que accidentalmente causé, no lo volvió a sugerir.

También ocupé la IA para que me dijera cuáles fechas se repetían y así correr la prueba necesaria.